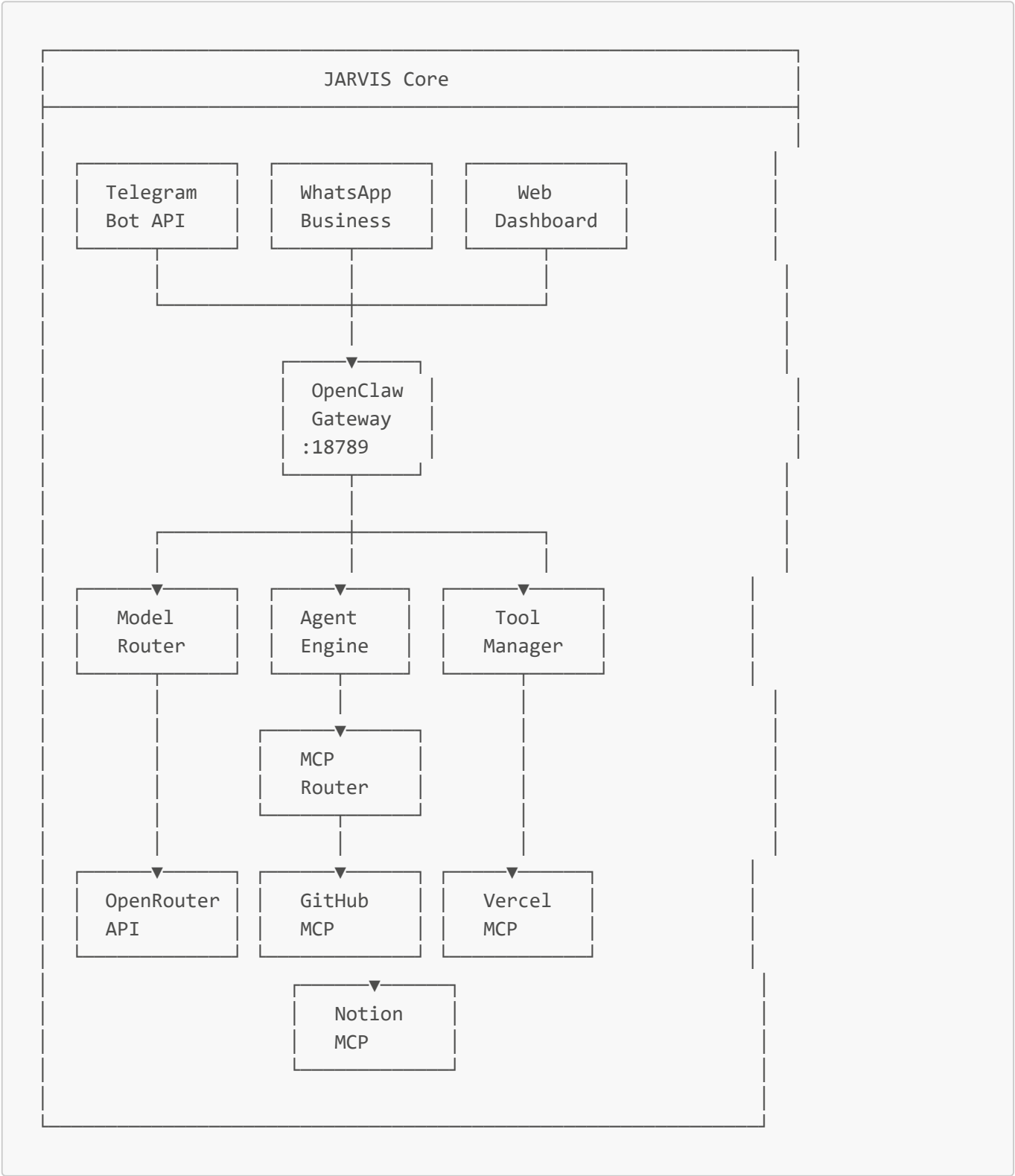


JARVIS Architecture Document

Version: 1.1 | **Last Updated:** 2026-04-01

System Overview

JARVIS is a self-hosted autonomous SWE workflow system built on a microservices architecture using Docker containers.



Component Details

1. OpenClaw Gateway

The central orchestrator that manages:

- **Message Routing** — Receives messages from Telegram/WhatsApp/Web
- **Model Selection** — Routes to appropriate model based on task type
- **Agent Orchestration** — Manages multi-step workflows
- **Tool Execution** — Calls MCP servers for integrations
- **State Management** — Maintains conversation context

Port: 18789 (HTTP) / 18790 (WebSocket) / 18791 (Browser control)

Configuration: `config/openclaw.json`

2. Model Router

Routes tasks to the optimal model:

```
routing:
  planning:
    primary: "openrouter/nvidia/nemotron-3-super-120b-a12b:free"
    fallback: ["glm-5-turbo", "minimax-m2.5:free"]

  coding:
    primary: "openrouter/nvidia/nemotron-3-super-120b-a12b:free"
    fallback: ["minimax-m2.5:free", "glm-5-turbo"]

  office:
    primary: "openrouter/minimax/minimax-m2.5:free"
    fallback: ["nemotron-3-super:free"]

  fast:
    primary: "glm-5-turbo"
    fallback: ["nemotron-3-super:free"]
```

3. MCP Servers

Individual Model Context Protocol servers for each integration:

Server	Container	npm Package	Purpose	Status
GitHub MCP	<code>jarvis-mcp-github</code>	<code>@modelcontextprotocol/server-github</code>	Repository management	☒ Configured
Vercel MCP	<code>jarvis-mcp-vercel</code>	<code>@vercel/mcp-server</code>	Deployment	☒ Configured

Server	Container	npm Package	Purpose	Status
Notion MCP	jarvis-mcp-notion	@notionhq/notion-mcp-server	Documentation	☒ Configured
Browser MCP	jarvis-mcp-browser	@anthropic/mcp-server-browser	Web automation	☒ Configured
Gmail MCP	jarvis-mcp-gmail	@modelcontextprotocol/server-gmail	Email	☐ Pending OAuth

4. Docker Architecture

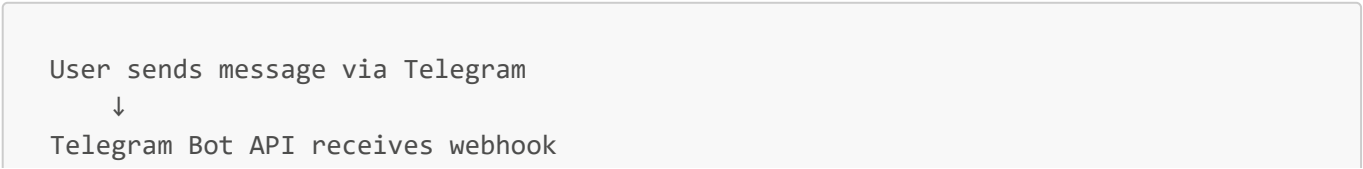
```
services:
  openclaw-gateway:
    image: openclaw:local
    ports:
      - "18789:18789"
      - "18790:18790"
      - "18791:18791"
    volumes:
      - ./config:/home/node/.openclaw
      - openclaw-data:/data
    networks:
      - jarvis-network

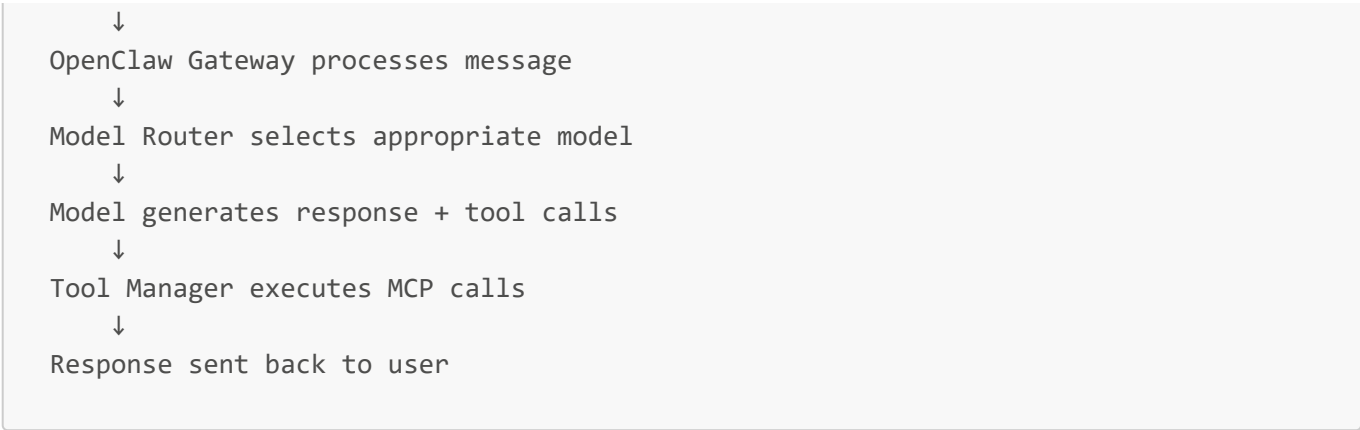
  mcp-github:
    image: node:20-slim
    command: npx -y @modelcontextprotocol/server-github
    environment:
      - GITHUB_TOKEN=${GITHUB_TOKEN}
    networks:
      - jarvis-network

  mcp-notion:
    image: node:20-slim
    command: npx -y @notionhq/notion-mcp-server
    environment:
      - NOTION_API_KEY=${NOTION_API_KEY}
    networks:
      - jarvis-network
```

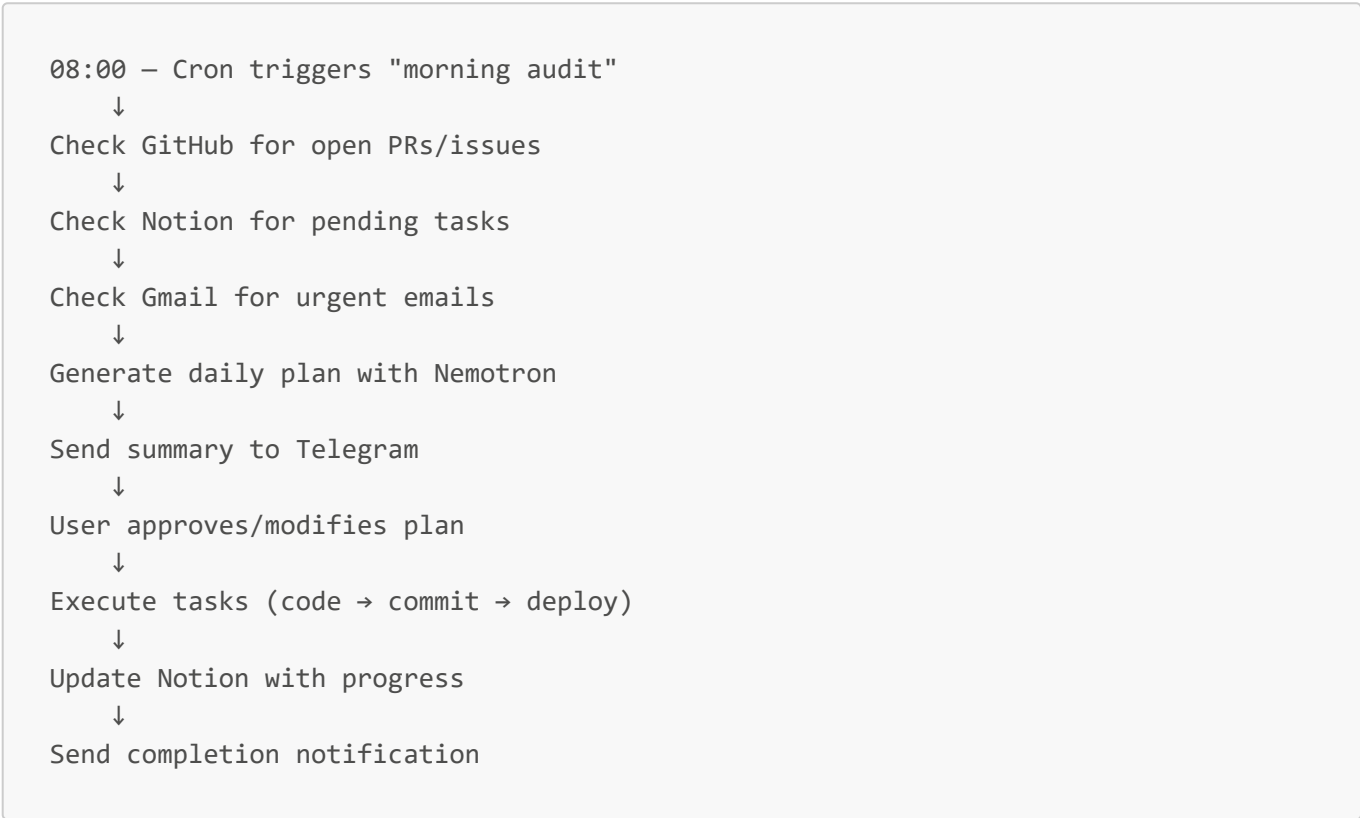
Data Flow

Message Processing Flow



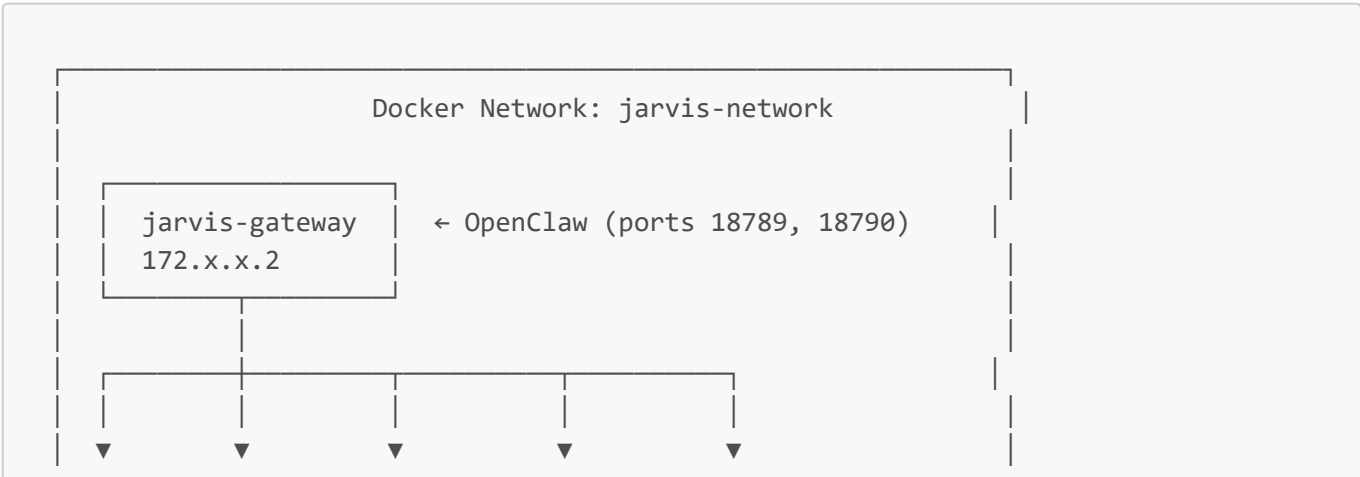


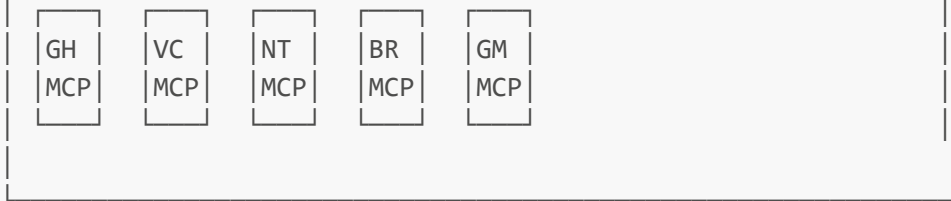
Daily Workflow Flow



MCP Integration Architecture

Network Topology



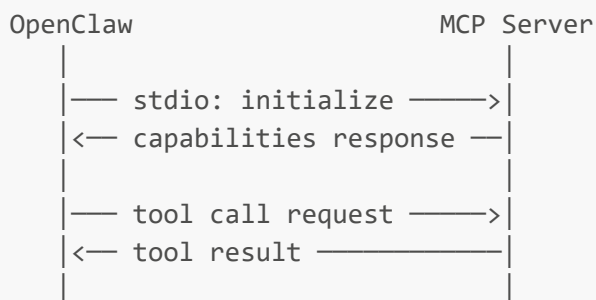


MCP Discovery

OpenClaw discovers MCP servers through:

1. **Configuration file** (`config/openclaw.json`) - defines MCP commands and env
2. **Docker network** - all MCPs on `jarvis-network` for internal communication
3. **npx execution** - MCPs run as child processes of OpenClaw

MCP Communication Protocol



Security Architecture

Authentication

- **Telegram:** Bot token + allowlist (user ID: 343865518)
- **Web Dashboard:** Token-based authentication
- **API Keys:** Stored in `.env` file (never in code)

Network Security

- Gateway binds to `0.0.0.0:18789` (configurable)
- MCP servers run on internal Docker network
- External access via reverse proxy (future)

Data Security

- All configs version-controlled in Git
 - Sensitive data in `.env` (gitignored)
 - Docker volumes for persistent data
 - VPS snapshots for backup
-

Configuration Files

File	Purpose
config/openclaw.json	Main OpenClaw gateway configuration
config/models.yaml	Model routing and fallback configuration
config/mcp-servers/github.yaml	GitHub MCP server configuration
config/mcp-servers/vercel.yaml	Vercel MCP server configuration
config/mcp-servers/notion.yaml	Notion MCP server configuration
config/mcp-servers/gmail.yaml	Gmail MCP server configuration
config/mcp-servers/browser.yaml	Browser MCP server configuration
.env	Environment variables (tokens, keys)

Scaling Strategy

Vertical Scaling (Current)

- KVM-4: 4 vCPU / 16GB RAM / 200GB NVMe
- Sufficient for single-user JARVIS

Horizontal Scaling (Future)

- Multiple JARVIS instances per client
- Load balancer for web dashboard
- Shared MCP servers across instances

Technology Stack

Layer	Technology	Version
Container Runtime	Docker	24.x
Orchestration	Docker Compose	2.x
Gateway	OpenClaw	Latest
LLM API	OpenRouter	-
MCP Protocol	Model Context Protocol	1.0
Version Control	Git	2.x
CI/CD	GitHub Actions	-
Deployment	Vercel (apps)	-
Documentation	Notion	-

This document describes the technical architecture of JARVIS v1.0. Refer to MASTER-PLAN.md for project roadmap and business decisions.